

Informe de seguimiento para Gerencia

Fecha: /06/2026 Nombre del proyecto: TAUROS

Descripción de la revisión: Número de revisión y descripción de alto nivel

Coordinador de la revisión: Reyna Elia Melara Abarca

Lista de participantes: Nombre de los integrantes del equipo que participan en la revisión	Actividades realizadas y Porcentaje de cumplimiento respecto a las actividades que tiene encomendadas	
	Actividad	%
Domínguez Jimenez Ana Andrea	Definición de stakeholders y el objetivo del producto.	30%
Rosales Rodríguez Irvin Axel	Diseño de la interfaz.	30%
Sánchez Delago Xcaret	Visión del producto y herramientas de HW y SW	30%
Zamora Gonzalez Aldo Hermilo	Definición de actores, modelo de contexto y modelo de contenedores.	10%

Acción recomendada:	
Aceptar el trabajo	()
Rechazar el trabajo	()
Necesario revisar nuevamente	()

Observaciones y acuerdos:
Se anotan comentarios generales, problemas detectados y acuerdos tomados.

Nombre y firma de conformidad de todos los participantes:

Aplicación web asistente de codificación con prompt para python con IA - TAURUS

Descripción del proyecto

Nombre tentativo: TAURUS(PythonLab Web o Ayudante Python Online)

Descripción general:

Plataforma web educativa e interactiva que permite a los usuarios aprender, practicar y desarrollar código Python directamente desde el navegador, sin necesidad de instalar nada localmente. Incluye un editor de código en línea con ejecución segura, un catálogo de ejercicios por niveles, un asistente inteligente para detectar errores y sugerir correcciones, y la capacidad de guardar proyectos personales. Está dirigida a principiantes que desean aprender Python y a programadores intermedios que buscan perfeccionar sus habilidades mediante práctica guiada y validación automatizada.

Objetivos generales

TAURUS se plantea como una plataforma accesible y segura que facilita el aprendizaje de Python directamente desde el navegador, con una interfaz intuitiva que integra área de prompts, historial y resaltado de sintaxis. Su propósito es ofrecer un entorno guiado en el que las respuestas se limiten a Python, generando código válido y alertando sobre posibles riesgos. El sistema guarda automáticamente las interacciones y permite gestionarlas, mientras controla el uso según planes gratuitos o premium. La seguridad se garantiza mediante sanitización de entradas, contraseñas encriptadas y protección de credenciales, y la integración con IA se asegura con conexiones estables y consistentes. Además, busca mantener un rendimiento rápido y escalable, soportando múltiples usuarios concurrentes, y promueve buenas prácticas de programación siguiendo estándares como PEP 8 y ofreciendo comparaciones y sugerencias de mejora.

Objetivos específicos

Brindar un entorno de ejecución Python 100% funcional en el navegador

Sin configuración local, seguro y con límites de recursos (sandbox).

Facilitar el aprendizaje práctico mediante ejercicios validados automáticamente

El usuario recibe corrección inmediata y sugerencias para mejorar.

Reducir la frustración inicial

Detectando errores comunes de sintaxis/lógica y ofreciendo explicaciones claras en español.

Permitir el seguimiento del progreso individual

Historial, puntuación, insignias y recomendación personalizada de ejercicios.

Garantizar un rendimiento y disponibilidad adecuados

Respuesta rápida (<2 segundos en interfaz, <3 segundos en ejecución) y alta disponibilidad (99.5%).

Ser seguro por diseño

Proteger al servidor de códigos maliciosos y los datos de los usuarios.

Ser escalable

Soportar cientos de usuarios concurrentes y permitir crecimiento sin rediseños profundos.

Promover buenas prácticas de programación

Comparador de soluciones, recomendaciones de estilo PEP 8 y documentación integrada.

DECLARACIÓN DE VISIÓN

TAURUS es una plataforma web educativa interactiva que permite a los usuarios aprender, practicar y desarrollar código Python directamente desde el navegador, sin necesidad de instalación local, mediante un asistente inteligente con IA que genera, optimiza y corrige código en español, con retroalimentación inmediata y seguimiento de progreso personalizado.

METÁFORA DEL ELEVADOR

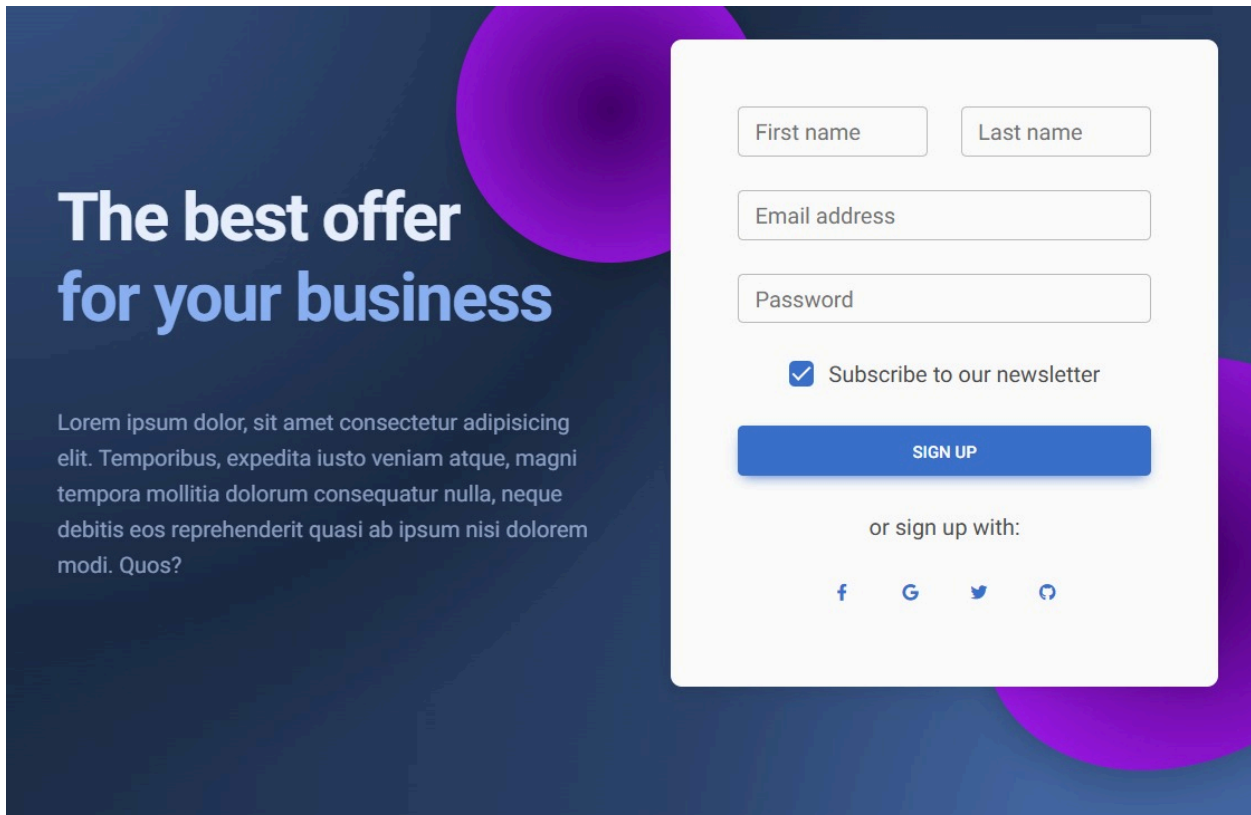
Para estudiantes y programadores que quieren aprender Python sin complicaciones, que necesitan un entorno práctico con retroalimentación inmediata y asistencia inteligente, TAURUS es una plataforma web educativa que permite escribir instrucciones en español, generar y optimizar código Python, y recibir corrección de errores al instante, a diferencia de los cursos tradicionales o editores locales,

nuestro producto ofrece un asistente con IA que entiende español, recuerda el contexto de la conversación y no requiere instalación local.

REQUERIMIENTOS

Especificación del Requerimiento funcional	Métrica
RF1. Registro e inicio de sesión	El usuario debe poder registrarse e iniciar sesión usando correo electrónico y contraseña. Se verificará con pruebas de autenticación exitosa y fallida (credenciales incorrectas).
RF2. Memoria contextual de la IA	Dentro de una misma sesión, la IA debe recordar instrucciones previas para responder preguntas de seguimiento. Se validará con una conversación de al menos 3 turnos dependientes del contexto.
RF3. Entrada de instrucciones en español	El sistema debe permitir al usuario escribir instrucciones en español para generar código Python. Se probará con prompts como "Genera una función que sume dos números".
RF4. Generación de código Python funcional	La IA debe devolver bloques de código Python que sean ejecutables y libres de errores sintácticos graves. Se verificará ejecutando el código generado en un entorno controlado.
RF5. Descarga o copia del código	El sistema debe permitir descargar el código generado en formato .py o copiarlo al portapapeles con un clic. Se probará simulando ambas acciones con éxito.
RF6. Optimización de código existente	El sistema debe recibir código Python existente y devolver una versión optimizada en legibilidad o eficiencia. Se medirá con métricas de tiempo de ejecución o reducción de complejidad ciclomática.
RF7. Corrección de errores en código	El usuario podrá ingresar código con errores (sintácticos o lógicos) y el sistema deberá corregirlos. Se validará con casos de error conocidos (ej. variable no definida, indentación incorrecta).

INTERFAZ



MODELOS

Requerimiento	¿Qué hay que guardar?	¿Por qué? (Persistencia)	Tabla y Campo
RF1. Registro e inicio de sesión	Email y contraseña del usuario	Para autenticar al usuario cada vez que vuelve	Usuario(email, password_hash)
RF2. Memoria contextual de IA	Conversación completa (lo que dice usuario y la IA)	Para que la IA recuerde los últimos mensajes dentro de la misma sesión	Historial_Actions(contenido, tipo, timestamp, id_sesion)

RF3. Entrada instrucciones español	El prompt que escribió el usuario (en español)	Para registro, depuración y mejora del modelo	Historial_Actions(contenido, tipo="entrada_usuario")
RF4. Generar código Python funcional	El código generado por la IA, junto al prompt que lo generó	Para ver qué código se generó a partir de qué instrucción	Historial_Actions(codigo_generado, contenido)
RF5. Descarga/copia del código	<i>No requiere persistencia</i>	Es una acción en tiempo real, no guarda datos	⚠ Solo UI, no tabla

PA-1: Registro exitoso (RF1)

Elemento	

Requerimiento	- Registro e inicio de sesión
Escenario	Usuario nuevo se registra correctamente
	1. Ir a pantalla de registro 2. Escribir email: estudiante@email.com 3. Escribir contraseña: MiClave123 4. Confirmar contraseña: MiClave123 5. Hacer clic en "Registrarse"
Datos de	mail nuevo no registrado antes
Resultado do	Mensaje: "Registro exitoso". Redirige a pantalla de login. El usuario a guardado en base de datos.

PA-2: Registro con email ya existente (RF1 - caso fallido)

Elemento	Valor
----------	-------

ID	PA-2
Requerimiento	RF1 - Registro e inicio de sesión
Escenario	Usuario intenta registrarse con email ya usado
Pasos	1. Ir a pantalla de registro 2. Escribir email: yaexiste@email.com (ya registrado) 3. Escribir contraseña: Cualquier123 4. Hacer clic en "Registrarse"
Datos de prueba	Email que ya está en la tabla Usuario
Resultado esperado	Mensaje de error: "El email ya está registrado". No se crea cuenta nueva.

PA-3: Login exitoso (RF1)

Elemento	Valor
----------	-------

ID	PA-3
Requerimiento	RF1 - Registro e inicio de sesión
Escenario	Usuario registrado inicia sesión correctamente
Pasos	1. Ir a pantalla de login 2. Escribir email: usuario@email.com (registrado) 3. Escribir contraseña: SuClave123 (correcta) 4. Hacer clic en "Iniciar sesión"
Datos de prueba	Credenciales correctas de usuario existente
Resultado esperado	Redirige al panel principal del asistente. Se crea una nueva sesión en tabla Sesion con estado "Activa".

PA-4: Login con contraseña incorrecta (RF1 - caso fallido)

Elemento	Valor
ID	PA-4
Requerimiento	RF1 - Registro e inicio de sesión
Escenario	Usuario escribe mal su contraseña

Pasos	1. Ir a pantalla de login 2. Escribir email: usuario@email.com (existente) 3. Escribir contraseña: ContraseñaIncorrecta 4. Hacer clic en "Iniciar sesión"
Datos de prueba	Email correcto, contraseña incorrecta
Resultado esperado	Mensaje de error: "Credenciales incorrectas". No se inicia sesión. No se crea sesión nueva.

PA-5: Memoria contextual de IA (RF2)

Elemento	Valor
ID	PA-5
Requerimiento	RF2 - Memoria contextual de IA
Escenario	La IA recuerda información de turnos anteriores dentro de la misma sesión

Pasos	1. Iniciar sesión (se crea sesión nueva) 2. Turno 1 - Usuario escribe: "Llama a la variable 'contador'" 3. IA responde: "Ok, usaré la variable 'contador'" 4. Turno 2 - Usuario escribe: "Crea una función que aumente esa variable en 1" 5. IA responde: "def aumentar(): global contador; contador += 1" (uso correcto de 'contador') 6. Turno 3 - Usuario pregunta: "¿Cómo se llamaba la variable que definimos?" 7. IA responde: "Se llamaba 'contador'"
Datos de prueba	3 turnos de conversación en la misma sesión
Resultado esperado	La IA responde consistentemente usando información de turnos anteriores. En la tabla Interaccion aparecen los 3 prompts del usuario y 3 respuestas de IA con el mismo id_sesion y timestamps ordenados.

PA-6: Entrada de instrucciones en español (RF3)

Elemento	Valor
ID	PA-6
Requerimiento	RF3 - Entrada de instrucciones en español
Escenario	Usuario escribe instrucción en español y la IA entiende

Pasos	1. Iniciar sesión 2. En el chat del asistente, escribir: "Genera una función que sume dos números y se llame sumar" 3. Enviar mensaje
Datos de prueba	Prompt en español: "Genera una función que sume dos números y se llame sumar"
Resultado esperado	IA genera código Python: <code>def sumar(a, b): return a + b</code> El prompt del usuario se guarda en <code>Interaccion.prompt_usuario</code> (español).

PA-7: Generación de código Python funcional (RF4)

Elemento	Valor
ID	PA-7
Requerimiento	RF4 - Generación de código Python funcional
Escenario	El código generado por IA se puede ejecutar sin errores

Pasos	1. Iniciar sesión 2. Prompt: "Genera una función que calcule el factorial de un número" 3. IA devuelve código 4. Copiar el código y ejecutarlo en Python (entorno controlado) 5. Probar factorial(5)
Datos de prueba	Prompt de factorial, ejecutar código generado
Resultado esperado	El código ejecuta sin errores. factorial(5) devuelve 120. El código generado se guarda en Interaccion.codigo_salida.

PA-8: Descarga de código en formato .py (RF5)

Elemento	Valor
ID	PA-8
Requerimiento	RF5 - Descarga o copia del código
Escenario	Usuario descarga el código generado como archivo .py
Pasos	1. Después de generar código (ej. PA-7) 2. Hacer clic en botón "Descargar .py" 3. Verificar archivo descargado

Datos de prueba	Código generado previamente
Resultado esperado	Se descarga un archivo codigo_generado.py con el código exacto que mostró la IA.

PA-9: Copiar código al portapapeles (RF5)

Elemento	Valor
ID	PA-9
Requerimiento	RF5 - Descarga o copia del código
Escenario	Usuario copia el código generado al portapapeles con un clic
Pasos	1. Después de generar código 2. Hacer clic en botón "Copiar" 3. Abrir editor de texto y pegar (Ctrl+V)
Datos de prueba	Código generado previamente
Resultado esperado	El código se pega exactamente igual en el editor de texto.

PA-10: Optimización de código existente (RF6)

Elemento	Valor
ID	PA-10
Requerimiento	RF6 - Optimización de código existente
Escenario	Usuario sube código ineficiente y la IA lo optimiza
Pasos	1. Iniciar sesión 2. Subir código ineficiente (buscar elemento en lista sin usar in): python def buscar(lista, elemento): for i in range(len(lista)): if lista[i] == elemento: return True return False 3. Prompt: "Optimiza este código" 4. IA devuelve versión optimizada
Datos de prueba	Código con complejidad $O(n)$ pero escrito de forma poco legible/eficiente
Resultado esperado	IA devuelve versión mejorada: python def buscar(lista, elemento): return elemento in lista Se guardan ambos códigos (original y optimizado) en Interaccion.codigo_entrada y Interaccion.codigo_salida.

PA-11: Corrección de error sintáctico (RF7)

Elemento	Valor
----------	-------

ID	PA-11
Requerimiento	RF7 - Corrección de errores en código
Escenario	Usuario sube código con error sintáctico (indentación incorrecta)
Pasos	1. Iniciar sesión 2. Subir código con error: python def hola(): print("hola") print("mundo") (el segundo print está mal indentado) 3. Prompt: "Corrige este código" 4. IA devuelve versión corregida
Datos de prueba	Código con error de indentación
Resultado esperado	IA devuelve: python def hola(): print("hola") print("mundo") (ambos prints dentro de la función).

PA-12: Corrección de error lógico (RF7)

Elemento	Valor
ID	PA-12
Requerimiento	RF7 - Corrección de errores en código

Escenario	Usuario sube código con error lógico (variable no definida)
Pasos	1. Iniciar sesión 2. Subir código con error: python def promedio(lista): suma = sum(lista) return total / len(lista) (usa total pero la variable se llama suma) 3. Prompt: "Corrige este código" 4. IA devuelve versión corregida
Datos de prueba	Código con variable no definida
Resultado esperado	IA devuelve: python def promedio(lista): suma = sum(lista) return suma / len(lista)

RX1. Entorno de ejecución. El código generado no se ejecutará en el servidor del backend.
RX2. El tamaño de los prompts está limitado por la IA utilizada.
RX3. Fecha de entrega. El desarrollo de la aplicación debe completarse dentro del calendario escolar establecido (semestre actual).

Reglas de negocio

RNE-01 Autenticación y sesión

RNE-01.1 Solo usuarios autenticados pueden acceder al asistente de codificación.

RNE-01.2 Las sesiones expiran después de 30 minutos de inactividad.

RNE-01.3 El registro requiere email válido y contraseña con mínimo 8 caracteres, 1 mayúscula y 1 número.

RNE-02 Gestión de prompts (consultas al asistente)

RNE-02.1 Cada prompt debe tener un lenguaje definido (Python) y una descripción clara.

RNE-02.2 El asistente solo genera respuestas para preguntas relacionadas con Python (sintaxis, librerías, lógica, errores).

RNE-02.3 Si el prompt no es sobre Python, el sistema responde: "Este asistente solo ayuda con Python".

RNE-03 Historial de conversaciones

RNE-03.1 Se guarda automáticamente cada prompt y su respuesta generada por IA.

RNE-03.2 El usuario puede ver, buscar y eliminar su historial.

RNE-03.3 El historial se ordena por fecha descendente (más reciente primero).

RNE-04 Generación de código

RNE-04.1 El código generado debe incluir sintaxis válida de Python 3.8+.

RNE-04.2 Si la IA no puede generar código seguro, debe mostrar advertencia y sugerir revisión manual.

RNE-04.3 El usuario puede copiar el código generado con un botón "Copiar".

RNE-05 Límites de uso

RNE-05.1 Usuarios gratuitos: máximo 50 prompts por día.

RNE-05.2 Usuarios premium: máximo 500 prompts por día.

RNE-05.3 Si se excede el límite, se muestra mensaje y se bloquea hasta el día siguiente.

RNE-06 Persistencia con MySQL y Sequelize

RNE-06.1 Modelos mínimos: User, Prompt, Response, UserLimit.

RNE-06.2 Toda interacción con la base de datos debe hacerse a través de Sequelize (evitar SQL directo).

RNE-06.3 Las contraseñas se almacenan hasheadas con bcrypt.

RNE-07: Frontend (React + Bootstrap)

RNE-07.1 La interfaz debe ser responsive (Bootstrap grid system).

RNE-07.2 El área de prompt debe tener un campo de texto y un botón "Enviar".

RNE-07.3 El resultado debe mostrarse en un bloque con formato de código resaltado (syntax highlighting).

RNE-07.4 Debe haber una sección “Historial” visible.

RNE-08 Backend (Node.js + Express)

RNE-08.1 Todos los endpoints deben responder en JSON.

RNE-08.2 Endpoints requeridos:

- ☐ POST /api/auth/login
- ☐
- ☐ POST /api/auth/register
- ☐
- ☐ POST /api/prompt (enviar prompt, obtener respuesta)
- ☐
- ☐ GET /api/history (obtener historial del usuario)
- ☐
- ☐ DELETE /api/history/:id

RNE-08.3 Los errores deben responder con código HTTP adecuado (400, 401, 403, 500) y mensaje claro.

RNE-09 Integración con IA

RNE-09.1 La IA se conecta vía API externa (ej. OpenAI o local con modelo de código abierto).

RNE-09.2 Si la API de IA falla, se reintentará 2 veces. Si persiste, se responde con error controlado.

RNE-09.3 El prompt enviado a la IA debe incluir un pre-prompt del sistema: “Eres un asistente experto en Python. Responde solo con código y breves explicaciones.”

RNE-10 Seguridad

RNE-10.1 Sanitizar todo prompt antes de enviarlo a la IA (evitar inyección de prompts maliciosos).

RNE-10.2 No mostrar errores internos del servidor al cliente.

RNE-10.3 Usar variables de entorno para claves de API y credenciales de BD.

RNE-11 Construcción (Webpack + Babel)

RNE-11.1 Webpack debe empaquetar React y Bootstrap.

RNE-11.2 Babel debe transpilar ECMAScript 6+ a ES5 compatible.

RNE-11.3 El entorno de desarrollo debe tener hot reload (webpack-dev-server).

BASIC FLOW

1. Usuario abre aplicación web (React + Bootstrap)
- 2.
3. Sistema carga pantalla de login/registro
- 4.
5. Usuario ingresa credenciales y autentica
- 6.
7. Sistema valida usuario y genera token JWT
- 8.
9. Sistema verifica límite diario de prompts en MySQL (Sequelize)
- 10.
11. Usuario escribe prompt sobre Python en el campo de texto
- 12.
13. Frontend valida que prompt no esté vacío
- 14.
15. Sistema envía prompt a backend Express vía POST /api/prompt
- 16.
17. Backend sanitiza prompt (protección contra inyección)
- 18.
19. Backend prepara pre-prompt: "Eres experto en Python. Responde solo con código y breve explicación"
20. Backend envía prompt + pre-prompt a API de IA
21. IA genera respuesta (código Python + explicación)
- 22.
23. Backend guarda prompt y respuesta en MySQL usando Sequelize
- 24.
25. Backend responde al frontend con JSON (código + explicación)
- 26.
27. Frontend muestra respuesta con resaltado de sintaxis (syntax highlighting)
- 28.
29. Usuario copia el código generado con botón "Copiar"
- 30.
31. Usuario puede ver su historial de prompts en el panel lateral

ALTERNATE FLOWS

- A1 Credenciales inválidas
- A2 Usuario no autenticado intenta acceder al asistente
- A3 Prompt vacío o con menos de 3 caracteres
- A4 Prompt no relacionado con Python (ej. JavaScript, HTML, etc.)
- A5 Límite diario de prompts excedido (50 para gratis, 500 para premium)
- A6 API de IA no responde o timeout
- A7 API de IA devuelve error (500, 429, etc.)
- A8 Código generado contiene palabras inseguras (eval, exec, import, os.system)
- A9 Error al guardar en MySQL (disco lleno, conexión perdida)
- A10 Token JWT expirado (sesión mayor a 30 minutos inactiva)
- A11 Webpack no compila el frontend (error en build)
- A12 Babel no transpila código ES6 a ES5 correctamente
- A13 Usuario intenta inyección de prompt malicioso (prompt injection)
- A14 Usuario intenta acceder a historial de otro usuario
- A15 Bootstrap no carga (falla CDN o CSS)
- A15 Bootstrap no carga (falla CDN o CSS)
- A16 Usuario solicita eliminar prompt del historial
- A17 Usuario solicita exportar historial a JSON/CSV
- A18 Usuario premium intenta exceder límite de 500 prompts
- A19 Usuario gratuito intenta usar funcionalidad premium
- A20 Backend Express lanza excepción no controlada (error 500)

CASO DE USO 1: Ver y gestionar historial de prompts

- Actor: Usuario autenticado
- Propósito: Visualizar prompts anteriores y sus respuestas generadas por IA
- Precondiciones: Usuario ha iniciado sesión. Existe al menos un prompt en historial.
- Postcondiciones: Ninguna (solo consulta) o historial modificado se elimina.

CASO DE USO 2: Autenticación y control de límites

- Actor: Usuario nuevo o existente (gratis o premium)
- Propósito: Registrar usuario, iniciar sesión y gestionar límites según su plan
- Precondiciones: Ninguna (para registro). Para login: usuario ya registrado.
- Postcondiciones: Usuario autenticado. Sesión activa con JWT. Contador de prompts diarios inicializado.

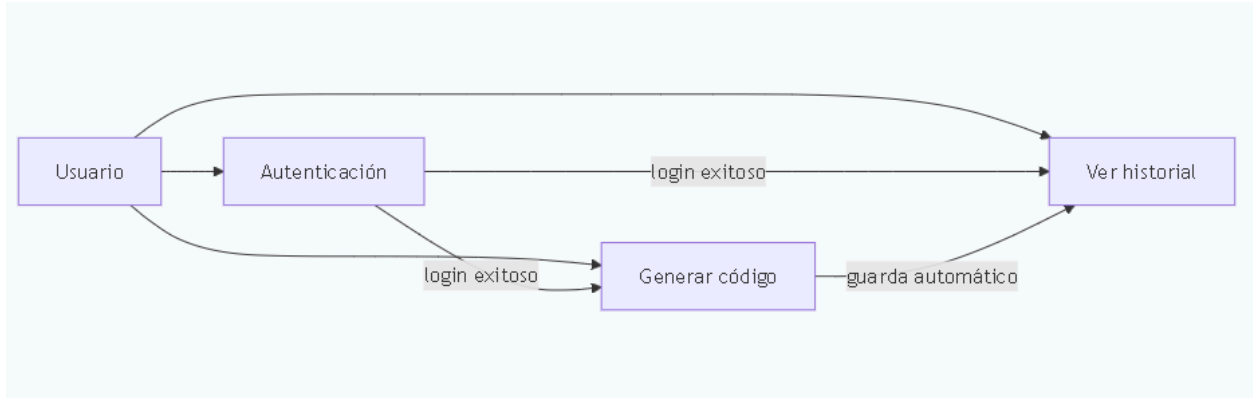


Diagrama de relación entre casos de uso con [Mermaid.js](#)

CASO DE USO 3: Instrucciones en español para generar código en Python

- Actor: Usuario autenticado.
- Propósito: Redactar prompts en español para generar código Python.
- Precondiciones: Sesión iniciada del usuario.
- Postcondiciones:

CASOS DE USO:

Actor Principal: Usuario (Gratuito / Premium)	CU1. Generar código Python desde prompt
Trayectoria Principal (Escenario Principal): El Caso de Uso comienza cuando el Usuario desea generar un bloque de código nuevo. 1. El Usuario escribe una instrucción en español solicitando código Python [RF3] . 2. El sistema verifica que el usuario no haya excedido su límite diario de prompts	Extensiones: Ext 1 – [RNE-05] Límite de prompts excedido: El sistema bloquea la entrada y muestra mensaje de límite alcanzado. Ext 2 – [RX2] El tamaño del prompt excede el límite permitido por la API.

(50 o 500) **[RNE-05]**.

3. El sistema agrupa la instrucción actual con el historial previo de la sesión **[RF2, RNE-03]**.

4. El sistema envía la solicitud al Servicio de IA y recibe la respuesta **[RNE-09]**.

5. El sistema muestra en pantalla un bloque de código Python funcional **[RF4]**.

6. El Usuario selecciona descargar el archivo `.py` o copiarlo al portapapeles **[RF5]**.

El Caso de Uso termina.

Fragmento (Slices):

Fragmento 1 – Generación simple (primer prompt de la sesión).

Fragmento 2 – Generación con contexto (La IA recuerda el prompt anterior de la misma sesión).

Ext 3 – El Servicio de IA (API Externa) no responde (Timeout 500/429).

Ext 4 – El usuario pide que el código se ejecute en la plataforma (El sistema lo deniega cumpliendo la restricción **[RX11]**).

Actor Principal: Usuario (Gratuito / Premium)	CU2. Optimizar y corregir código ingresado
Trayectoria Principal (Escenario Principal): El Caso de Uso comienza cuando el Usuario tiene un código propio que necesita mejorar o arreglar. 1. El Usuario pega un bloque de código Python existente en el editor del sistema. 2. El Usuario selecciona la opción "Optimizar legibilidad/eficiencia" [RF6] o "Corregir errores" [RF7]. 3. El sistema valida la sesión y contabiliza el prompt en el límite diario del usuario [RNE-05]. 4. El sistema procesa el código a través de la API de IA. 5. El sistema muestra el código refactorizado/corregido y una breve explicación de los cambios. 6. El Usuario descarga el resultado en .py o lo copia [RF5]. El Caso de Uso termina.	Extensiones: Ext 1 – El código pegado está en un lenguaje distinto a Python (El sistema devuelve un error de validación). Ext 2 – El código ingresado contiene datos sensibles, el sistema lo procesa pero la base de datos debe encriptarlo [RNF5]. Ext 3 – La IA no encuentra errores en el código ingresado (Notifica que el código ya es óptimo).

Fragmentos (Slices): Fragmento 1 – Identificación y solución de errores de sintaxis (Corrección). Fragmento 2 – Refactorización para reducir complejidad ciclomática (Optimización).	
--	--

Actor Principal: Visitante / Usuario	CU3. Iniciar sesión y validar cuenta
Trayectoria Principal (Escenario Principal): El Caso de Uso comienza cuando un visitante desea acceder a las herramientas del Asistente IA. 1. El visitante ingresa a la página principal del sistema (Frontend React) [RNE-07]. 2. El visitante proporciona su email y contraseña [RF1]. 3. El sistema verifica las credenciales en la base de datos MySQL [RNE-06]. 4. El sistema determina si el usuario tiene rol "Gratuito" o "Premium" y carga sus límites de uso [RNE-01, RNE-05]. 5. El sistema redirige al panel de control de prompts. El Caso de Uso termina. Fragmentos (Slices): Fragmento 1 – Inicio de sesión exitoso de usuario existente. Fragmento 2 – Registro y creación de un nuevo Usuario Gratuito.	Extensiones: Ext 1 – Credenciales incorrectas (Contraseña o email no coinciden). Ext 2 – Intento de registro con un correo electrónico que ya existe en la base de datos. Ext 3 – El sistema experimenta alta concurrencia superando los 50 usuarios simultáneos [RNF7] (Manejo de cola de espera).

1. Gestión de Límites de Uso (Relacionada con CU1 - Ext 1)

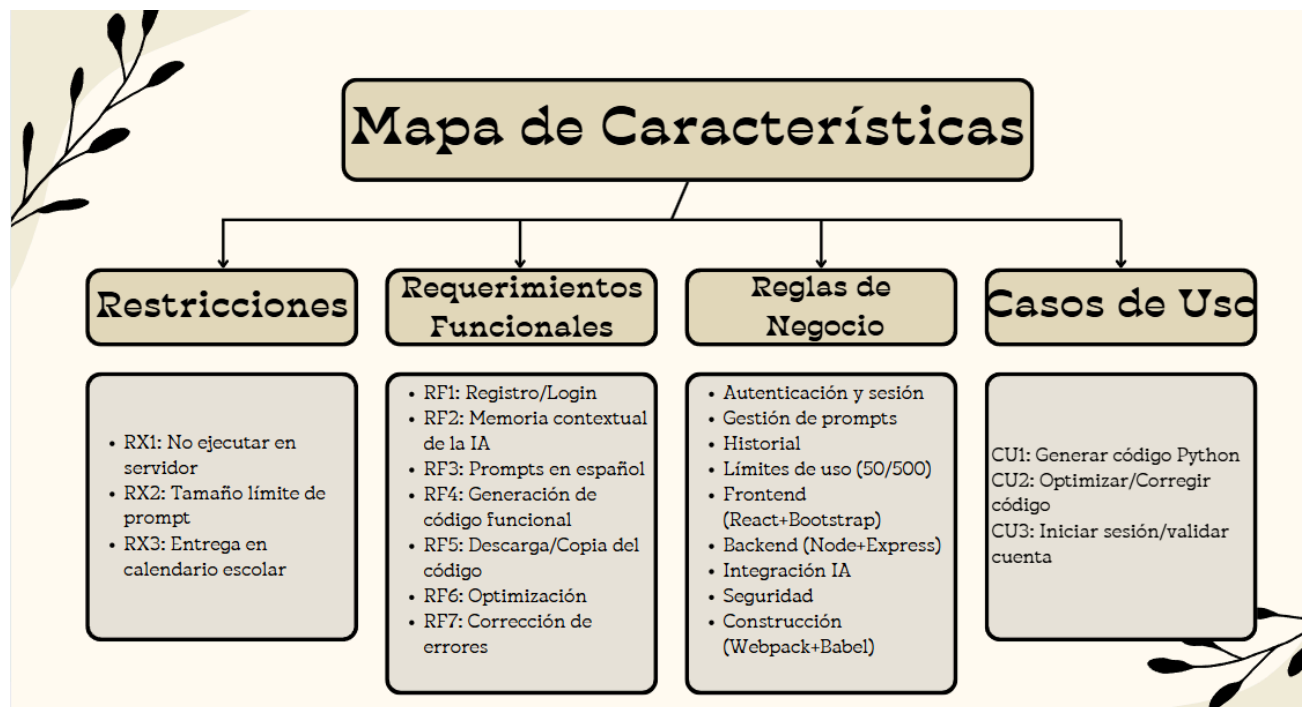
ID: HU_TAU_02	Control de cuota diaria de prompts
Descripción	Yo como Usuario Gratuito, quiero recibir un aviso claro cuando alcance mi límite de 50 prompts diarios, de modo que entienda por qué no puedo generar más código y considere adquirir una cuenta Premium.
Criterios de Aceptación	1. El sistema debe consultar el contador en la base de datos MySQL antes de enviar el prompt a la IA. 2. Si el contador = 50, se debe deshabilitar el botón de "Enviar". 3. Mostrar un mensaje: "Has alcanzado tu límite diario. Vuelve mañana o mejora tu plan".

2. Restricción de Ejecución de Código (Relacionada con CU1 - Ext 4)

ID: HU_TAU_03	Denegación de ejecución en servidor
Descripción	Yo como Administrador del Sistema, quiero que el asistente informe al usuario que no es posible ejecutar código en la plataforma, de modo que se mantenga la seguridad del backend y se cumplan las restricciones del proyecto.
Criterios de Aceptación	1. Si el usuario ingresa un prompt como "ejecuta este código", la IA debe responder explicando que el sistema es solo para generación y optimización. 2. No deben existir terminales ni consolas de ejecución en la interfaz de React.

3. Manejo de Fallos de API Externa (Relacionada con CU1 - Ext 3)

ID: HU_TAU_05	Notificación de indisponibilidad de la IA
Descripción	Yo como Usuario del sistema, quiero recibir una notificación si el servicio de IA externo está caído o lento, de modo que no piense que el error es de mi conexión o de la aplicación TAUROS.
Criterios de Aceptación	1. El backend debe capturar errores 500 o 429 de la API externa. 2. El frontend debe mostrar un "Toast" o alerta: "El servicio de IA no está disponible temporalmente. Inténtalo de nuevo en unos minutos".



Stakeholders

Usuarios finales (principiantes e intermedios en Python):

- Buscan aprender, practicar y mejorar sus habilidades en Python.
- Necesitan retroalimentación inmediata, historial de prompts y un entorno seguro.

Usuarios gratuitos:

- Acceso limitado (50 prompts/día).
- Representan la base de adopción inicial.

Usuarios premium:

- Acceso extendido (500 prompts/día).
- Esperan mayor disponibilidad, soporte y funcionalidades avanzadas.

Equipo de desarrollo (Frontend, Backend, DevOps):

- Implementan la plataforma con React, Bootstrap, Node.js, Express, Sequelize y MySQL.
- Garantizan seguridad, escalabilidad y rendimiento.

Equipo de IA / Integración:

- Gestiona la conexión con APIs externas (OpenAI u otros modelos).
- Asegura que las respuestas sean relevantes, seguras y en español.

Administradores del sistema:

- Supervisan usuarios, límites, seguridad y disponibilidad.
- Gestionan credenciales, claves y configuración de infraestructura.

Instituciones educativas / formadores:

- Pueden usar TAURUS como herramienta de apoyo en cursos de programación.

Herramientas de Hardware y Software

Hardware (Requerimientos mínimos para desarrollo y despliegue)

componente	especificación	justificación
Servidor Backend	CPU: 2 vCPUs, RAM: 4 GB, Disco: 20 GB SSD	Suficiente para Node.js + Express + MySQL con

		hasta 100 usuarios concurrentes. Permite crecimiento sin rediseño.
Servidor Base de Datos	CPU: 1 vCPU, RAM: 2 GB, Disco: 20 GB SSD	MySQL con Sequelize requiere recursos moderados. El tamaño estimado de datos (usuarios, historial) es bajo al inicio.
Equipo de Desarrollo	CPU: Intel i5 / AMD Ryzen 5, RAM: 8 GB, Disco: 256 GB SSD	Permite ejecutar entorno local con React (webpack-dev-server), Node.js, MySQL y herramientas de depuración sin lentitud.
Red	Ancho de banda mínimo: 10 Mbps	Suficiente para comunicación frontend-backend y consumo de API de IA externa.

Software

Categoría	Herramienta	Versión	Justificación
Frontend	React	18.x	Biblioteca declarativa y eficiente. Permite construir SPA interactiva con actualización en tiempo real del asistente y editor de código.
Frontend UI	Bootstrap	5.x	Framework CSS responsive. Garantiza interfaz adaptable a cualquier dispositivo (RNE-07.1) sin diseñar desde cero.

Frontend Build	Webpack	5.x	Empaqueta React, Bootstrap y assets. Requerido explícitamente por RNE-11.1.
Frontend Transpilador	Babel	7.x	Transpila ES6+ a ES5 para compatibilidad con navegadores antiguos (RNE-11.2).
Backend	Node.js	18.x o superior	Entorno JavaScript del lado del servidor. Ideal para APIs REST, integración con IA y ORM como Sequelize.
Backend Framework	Express	4.x	Framework minimalista para Node.js. Maneja rutas, middlewares y respuestas JSON según RNE-08.
Base de Datos	MySQL	8.x	Base de datos relacional robusta. Requerida por RNE-06.1 para modelos User, Prompt, Response, UserLimit.
ORM	Sequelize	6.x	ORM para Node.js con MySQL. Previene SQL injection y centraliza la lógica de persistencia (RNE-06.2).
Autenticación	JWT (jsonwebtoken)	9.x	Tokens para manejo de sesiones stateless. Permite verificar autenticación en cada petición.

Hashing	bcrypt	5.x	Almacena contraseñas hasheadas. Cumple RNE-06.3 (contraseñas encriptadas).
IA / LLM	OpenAI API / Modelo open source (ej. Mistral, Llama)	-	Provee la inteligencia para generar, optimizar y corregir código. Se conecta vía API externa según RNE-09.1.
Control de Versiones	Git + GitHub/GitLab	-	Permite trabajo colaborativo, trazabilidad de cambios y entrega del repositorio (requerido en tabla).
Entorno Desarrollo	webpack-dev-server	4.x	Hot reload para desarrollo ágil (RNE-11.3). Refleja cambios al instante sin recargar manualmente.
Gestor de Paquetes	npm / yarn	-	Maneja dependencias de frontend y backend (React, Express, Sequelize, etc.).

La selección tecnológica obedece a los requerimientos funcionales y no funcionales del proyecto:

- React + Bootstrap: Interfaz moderna, responsive y con resaltado de sintaxis (RNE-07).

-
- Node.js + Express: Backend ligero que maneja peticiones JSON y se integra fácilmente con APIs de IA.
-
- MySQL + Sequelize: Persistencia confiable con modelos claros y seguridad por defecto.
-
- JWT + bcrypt: Autenticación segura sin mantener estado en el servidor.
-
- Webpack + Babel: Build profesional que cumple con los estándares de entrega del curso.

Definición de Actores del Sistema

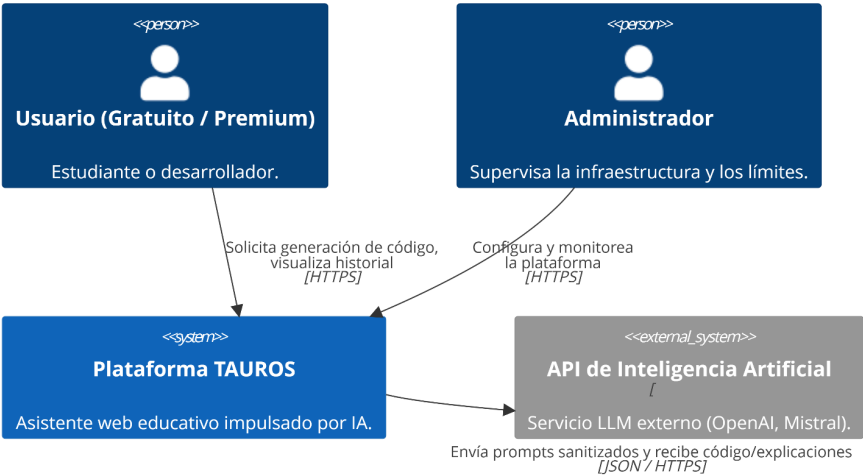
Esta tabla define quién y qué interactúa con **TAUROS**. Incluye tanto a los usuarios humanos como a los sistemas externos, basados en el análisis de stakeholders y requerimientos.

Actor	Tipo	Descripción y Responsabilidades
Usuario Gratuito	Humano (Principal)	Estudiante o principiante en programación que busca aprender Python. Su acceso está limitado a 50 prompts diarios. Interactúa con el sistema para generar, corregir y optimizar código.
Usuario Premium	Humano (Principal)	Programador intermedio o avanzado que requiere mayor capacidad de uso. Su límite es de 500 prompts diarios. Busca retroalimentación rápida y herramientas de refactorización.

Administrador del Sistema	Humano (Secundario)	Personal técnico o equipo de desarrollo que supervisa la plataforma. Gestiona usuarios, monitorea límites, seguridad, disponibilidad e infraestructura de la base de datos y la API.
API de IA Externa	Sistema Externo	Servicio LLM (como OpenAI, Llama o Mistral) que proporciona la "inteligencia". Recibe el <i>pre-prompt</i> del sistema junto con la instrucción del usuario y devuelve código Python válido con su explicación.

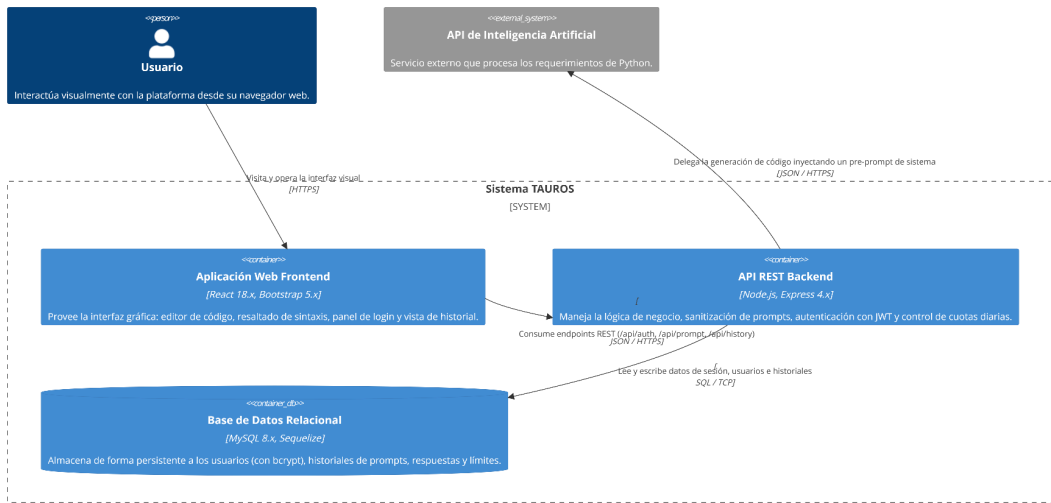
Modelo de Contexto (C4 - Nivel 1)

Diagrama de Contexto de Sistema (Nivel 1) - Proyecto TAUROS



Modelo de Contenedores (C4 - Nivel 2)

Diagrama de Contenedores (Nivel 2) - Proyecto TAUROS



Referencias

DeepSeek
DeepSeek. (2026). DeepSeek [modelo de inteligencia artificial]. DeepSeek AI.
<https://www.deepseek.com>

Gemini

Google. (2026). Gemini [modelo de inteligencia artificial]. Google DeepMind.
<https://deepmind.google>

Copilot

Microsoft. (2026). Copilot [asistente de inteligencia artificial]. Microsoft Corporation.
<https://copilot.microsoft.com>

Claude

Anthropic. (2026). Claude [modelo de inteligencia artificial]. Anthropic.
<https://www.anthropic.com>

Peredo Valderrama, R. (s.f.). Tecnologías para el desarrollo de aplicaciones web. Unidad IV: Desarrollo del lado cliente (3.4 Buenas prácticas). ESCOM – IPN, México.

Peredo Valderrama, R. (s.f.). Tecnologías para el desarrollo de aplicaciones web. Unidad I (1.5.2 Arquitecturas SOAP). ESCOM – IPN, México.

Peredo Valderrama, R. (s.f.). Tecnologías para el desarrollo de aplicaciones web. Unidad III (3.4.1 Responsividad). ESCOM – IPN, México.

Peredo Valderrama, R. (s.f.). Tecnologías para el desarrollo de aplicaciones web. Unidad I (1.1 Contextualización al desarrollo web). ESCOM – IPN, México.

Peredo Valderrama, R. (s.f.). Tecnologías para el desarrollo de aplicaciones web. Unidad I (1.6.1 Control de versiones). ESCOM – IPN, México.

Peredo Valderrama, R. (s.f.). *Tecnologías para el desarrollo de aplicaciones web. Unidad II (2.5.3 Documentación de servicios)*. ESCOM – IPN, México.

Cristiá, M. (2026). Diseño de software. FCEIA – Universidad Nacional de Rosario.

Massachusetts Institute of Technology. (s.f.). Develop your elevator pitch. MIT Career Advising & Professional Development. Recuperado de
<https://capd.mit.edu/resources/develop-your-elevator-pitch/?ct=1782149425333>

Viewpoints and Perspectives. (s.f.). *Stakeholders*. Recuperado de
<https://www.viewpoints-and-perspectives.info/home/stakeholders/>

Peredo Valderrama, R. (s.f.). Tecnologías para el desarrollo de aplicaciones web. Unidad III: Desarrollo del lado cliente (3.4.4 Manejo de recursos). ESCOM – IPN, México.